

Trabajo de Título

Etapa 2: análisis y diseño de Placement y Glance

Paolo Castañon Barrera

Juan Claudio Cárdenas Uribe
Profesor guía: Alejandro Mellado Gatica

29 de septiembre de 2026

Resumen

Se presenta el análisis verificable de OpenStack Placement y Glance para una reimplementación acotada en Go. Se preparó un laboratorio DevStack reproducible, scripts de verificación y captura, contratos API, modelos de datos y estrategias de compatibilidad. El host disponible no es un servidor dedicado Ubuntu 24.04; por seguridad DevStack no se instaló y no existe aún baseline funcional. El estado del backlog refleja esa limitación.

Contenido

1	Introducción	2
2	Estado respecto de Etapa 1	2
3	Retroalimentación recibida	2
4	Objetivos del Sprint 2	2
5	Entorno experimental	2
6	DevStack	2
7	Arquitectura general	2
8	Diseño Placement	2
9	Diseño Glance	3
10	Contratos API	3
11	Modelos de datos	3
12	Arquitectura objetivo Go	3
13	Estrategia de compatibilidad	3
14	Estrategia de pruebas	3
15	Decisiones técnicas	3
16	Estado del backlog	4
17	Riesgos	4
18	Conclusiones de la etapa	4
19	Próximos pasos Sprint 3	4

1 Introducción

El proyecto compara compatibilidad funcional y, posteriormente, rendimiento y recursos de servicios originales Python frente a implementaciones Go. No se presupone una mejora de rendimiento. La planificación oficial es el libro `Planificacion_Trabajo_Titulo_OpenStack_Placement_Glance.xlsx`, con fecha final de Sprint 2 el 2 de octubre de 2026.

2 Estado respecto de Etapa 1

El Excel contiene OE1–OE5, RF01–RF12, RNF01–RNF07 y backlog B01–B26. La Etapa 2 prioriza OE1 (análisis) y OE2 (entorno y baseline). En el repositorio inicial sólo existía un README; se incorporó una copia inalterada del libro y una matriz de requerimientos.

3 Retroalimentación recibida

No se encontró retroalimentación formal del profesor guía entre los insumos disponibles. `feedback-etapa1.md` deja el ítem pendiente para incorporación manual, sin inventar observaciones.

4 Objetivos del Sprint 2

OE1 exige arquitectura, APIs, dependencias y flujos críticos de Placement y Glance. OE2 exige OpenStack de un nodo reproducible, smoke tests y baseline original. RF03–RF10 describen implementación posterior; el diseño de esta etapa no los valida.

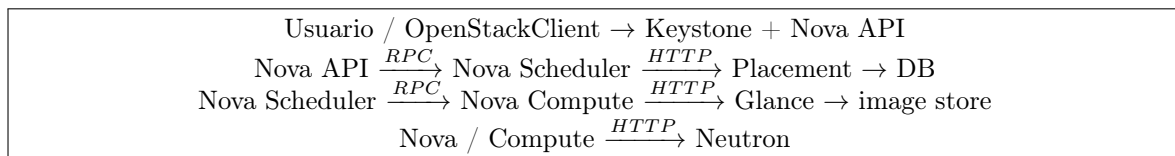
5 Entorno experimental

La auditoría del host registró CachyOS rolling, 16 CPU, 15 GiB RAM, 82 GiB libres, red disponible, virtualización no detectada y sudo no interactivo ausente. Se trata como estación de trabajo. La guía oficial recomienda Ubuntu 24.04 dedicado para DevStack y advierte cambios sustanciales al sistema. Por tanto no se ejecutó `stack.sh`.

6 DevStack

El instalador exige Ubuntu 24.04, usuario no root, declaración de VM dedicada, recursos y red. Fija DevStack `stable/2026.2` al SHA `0fb9685c1bf22cd0b4d576d7a74f64195ad934fe`. Usa el grafo de servicios por defecto (Keystone, Nova, Placement, Glance, Neutron, DB y RabbitMQ). Los SHAs y versiones reales de los servicios se registrarán después de instalar; hoy constan como UNINSTALLED. Credenciales van en `.env` ignorado por Git.

7 Arquitectura general



Las fuentes Mermaid completas están en `docs/sprint2/diagrams`. Las rutas HTTP utilizan tokens Keystone; las relaciones internas de Nova usan RPC.

8 Diseño Placement

El código upstream `stable/2026.2`, SHA `f4a89d3803cb23df22b8714569889ab39c918c34`, muestra handlers WSGI, microversiones, dominio y SQLAlchemy. ResourceProvider publica capacidad, Inventory cuantifica recursos, Trait y ResourceClass describen cualidades/tipos, y Allocation/Consumer registran uso. `generation` controla concurrencia y puede producir 409 ante actualización obsoleta. Falta validar pipeline y configuración de un proceso instalado.

9 Diseño Glance

El código upstream SHA 8cd693aad9d5982d9d71b4c34aab4de937ff70c separa API, dominio, DB de metadata y `glance_store` para binarios. El ciclo mínimo es reservar imagen, cargar datos, consultar, modificar metadata, descargar y borrar. El backend real del laboratorio todavía no se conoce; no se supone ruta de filesystem.

10 Contratos API

Servicio	Operaciones priorizadas	Estado de prueba
Placement	providers, inventories, traits, classes, allocations, candidates	Documentadas; sin baseline
Glance	POST/GET/PATCH/DELETE imágenes; PUT/GET file	Documentadas; sin baseline

La rama Placement declara 1.0–1.39. El cliente Nova inspeccionado solicita candidatos con 1.36 y usa 1.28 para operaciones que requieren consumer generation; la negociación efectiva no fue medida. Cada método/ruta, error y prioridad está en los inventarios API del repositorio.

11 Modelos de datos

Dominio	Entidades verificadas en modelos SQLAlchemy
Placement	ResourceProvider, Inventory, ResourceClass, Trait, Allocation, Consumer; relaciones y generaciones
Glance	Image, ImageProperty, ImageLocation, ImageTag, ImageMember; metadata en DB y bytes en store

Los diagramas ER indican claves y relaciones lógicas. En Placement no todas las relaciones ORM están declaradas como FK SQL.

12 Arquitectura objetivo Go

Se proponen procesos `cmd/placement` y `cmd/glance`; paquetes internos API, servicio, modelo y repositorio, más `glance/store`. Autenticación Keystone, configuración, middleware y logging son compartidos. Handlers negocian HTTP/microversión; servicios resuelven reglas; repositorios manejan transacciones; store transmite bytes.

13 Estrategia de compatibilidad

Cada petición se enviará al original y a Go con el mismo estado inicial. Se compararán status, JSON, tipos, campos, headers estables, errores y efectos persistidos. Nova y el cliente OpenStack serán verificadores adicionales. UUID, timestamps, orden no garantizado y request IDs se normalizarán sin ocultar diferencias semánticas. No hay fixtures ejecutadas aún.

14 Estrategia de pruebas

Unitarios Go, contrato API, comparación diferencial, integración con Nova y end-to-end de VM forman la pirámide. En la fase posterior se medirán p50/p95/p99 de latencia, throughput, CPU, RAM y errores con carga controlada. No se presentan benchmarks ficticios.

15 Decisiones técnicas

Los ADR en `design-decisions.md` comparan schema original, propio e intermedio. Se propone schema propio Placement compatible por API y metadata Glance acotada con store filesystem único, condicionado a la configuración real. Auth Keystone se conservará. Estas decisiones se revisarán contra el baseline.

16 Estado del backlog

ID	Estado	Motivo principal
B01	Bloqueado	falta VM dedicada
B02	Bloqueado	instalación no ejecutada
B03	Bloqueado	no hay servicios para smoke tests
B04	En progreso	código analizado; no servicio real
B05	En progreso	inventario preliminar; sin microversiones probadas
B06	Bloqueado	no hay tráfico Nova observado
B19	En progreso	diseño upstream; backend real pendiente

Ningún elemento S2 cumple aún todo su criterio. Avance formal: 0 de 7 elementos y 0 de 33 story points S2, es decir 0% por la métrica de aceptación del Excel.

17 Riesgos

Riesgos: falta de VM, diferencias entre código upstream y despliegue, microversiones negociadas no observadas, almacenamiento Glance real desconocido y retroalimentación formal no recibida. Cada uno tiene verificación pendiente en BLOCKERS.md.

18 Conclusiones de la etapa

Quedaron preparados artefactos de diseño y automatización, pero OE1 y OE2 no pueden declararse cumplidos sin contrastar flujos y APIs en DevStack. La ausencia de baseline impide cerrar Sprint 2 de manera verificable. El Excel actualizado reflejará bloqueos y avances parciales.

19 Próximos pasos Sprint 3

Primero provisionar la VM y completar B01–B03/B06 y baseline de ambos servicios. Después fijar contratos observados, implementar B07 (providers/inventories), B08 (traits/classes) y B11 (persistencia) con pruebas diferenciales; mantener B20/B25 de Glance en la planificación existente, sin adelantar implementación sin baseline.

References

- [1] <https://docs.openstack.org/devstack/latest/>
- [2] <https://docs.openstack.org/api-ref/placement/>
- [3] <https://docs.openstack.org/api-ref/image/v2/>
- [4] <https://docs.openstack.org/glance/latest/contributor/architecture.html>
- [5] <https://docs.openstack.org/nova/latest/admin/architecture>